

Universidade Federal do Rio Grande do Norte
Escola de Ciências e Tecnologia
IX Semana de Ciências e Tecnologia
Minicurso de Introdução a Banco de Dados

SQL e PostgreSQL

Parte 03

Breno Santana Santos
⟨breno.santos@ufrn.br⟩

11 de novembro de 2026

Sumário

1 Restrições de Integridade
- Chave Primária
- Chave Única
- Chave Estrangeira
- Restrição de Nulidade
- Restrição de Valor Padrão
- Restrição de Verificação

Restrições de Integridade

- É uma expressão booleana que está associada a algum BD e que precisa ser avaliada o tempo todo como **TRUE**.
- As restrições devem ser declaradas formalmente para que o SGBD possa monitorar todas as requisições de atualização e rejeitar as que violarem tais restrições, a fim de garantir a integridade e consistência dos dados em um BD.
- O SGBD verifica se a restrição atende ao BD, se atender, será adicionada ao catálogo e imposta daqui em diante, mas, caso contrário, será rejeitada.
- As restrições podem ser definidas nos comandos **CREATE TABLE** ou **ALTER TABLE**.

Restrições de Integridade

- Na SQL, as principais restrições de integridade são:
  - Chaves Candidatas:
    - Chave Primária (**PRIMARY KEY**);
    - Chave Única (**UNIQUE KEY**).
  - Chave Estrangeira (**FOREIGN KEY**);
  - Restrição de Nulidade (**NOT NULL**);
  - Restrição de Valor Padrão (**DEFAULT**);
  - Restrição de Verificação (**CHECK**).

Restrições de Integridade

Chave Primária

- Não permite entrada nula e todas as entradas são únicas, ou seja, a coluna deve conter valores não nulos e únicos.
- Garante a identificação única de um registro.
- Uma tabela pode conter apenas uma coluna como chave primária.
- Na prática, cria-se uma coluna que armazenará o código de identificação de um registro dentro de uma tabela.
- Utiliza-se a cláusula **PRIMARY KEY** para definir a chave primária de uma tabela.

- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  endereco VARCHAR(50) NOT NULL
);

CREATE TABLE tb_aluno (
  matricula INT NOT NULL,
  nome VARCHAR(50) NOT NULL,
  endereco VARCHAR(50) NOT NULL,
  PRIMARY KEY(matricula)
);
```

- Devemos nomear as restrições (**CONSTRAINTS**).
- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL,
  nome VARCHAR(50) NOT NULL,
  endereco VARCHAR(50) NOT NULL,
  CONSTRAINT pk_aluno PRIMARY KEY(matricula)
);
```

- Podemos definir uma chave primária composta, ou seja, chave primária formada por mais de uma coluna.
- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL,
  nome VARCHAR(50) NOT NULL,
  endereco VARCHAR(50) NOT NULL,
  cpf VARCHAR(11) NOT NULL,
  PRIMARY KEY(matricula, cpf)
);
```

- Podemos definir uma chave primária utilizando o comando **ALTER TABLE**.
- **Sintaxe:** `ALTER TABLE tb_nome ADD CONSTRAINT nome_restrição PRIMARY KEY(colunas_sep_vírgula);`
- **Exemplo:**
  - `ALTER TABLE tb_aluno ADD CONSTRAINT pk_aluno PRIMARY KEY(matricula);`
  - `ALTER TABLE tb_aluno ADD CONSTRAINT pk_aluno PRIMARY KEY(matricula, cpf);`

- Podemos remover uma restrição de chave primária por meio do comando **ALTER TABLE**.
- **Sintaxe:** `ALTER TABLE tb_nome DROP CONSTRAINT nome_restrição;`
- **Exemplo:** `ALTER TABLE tb_aluno DROP CONSTRAINT pk_aluno;`
- **Observação:** é recomendado (boa prática) o uso de chave primária, mas seu uso é facultativo.

Restrições de Integridade

Chave Única

- Aplicada a uma coluna para garantir que não haja nenhum valor duplicado nela, ou seja, garante a unicidade de um registro.
- Permite definir outras chaves.
- Permite valores nulos, mas admite apenas um único valor nulo.
- Podemos ter quantas colunas forem necessárias com a restrição **UNIQUE**.
- Utiliza-se a cláusula **UNIQUE** para definir uma chave única de uma tabela.

- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cpf VARCHAR(11) NULL UNIQUE
);

CREATE TABLE tb_aluno (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cpf VARCHAR(11) NULL,
  CONSTRAINT uk_aluno UNIQUE(cpf)
);
```

- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cpf VARCHAR(11) NULL,
  UNIQUE(cpf)
);
```

- Podemos definir uma chave única composta, ou seja, chave única formada por mais de uma coluna. Para isso, devemos definir a restrição **UNIQUE** em um índice composto pelas colunas desejadas ou especificar as colunas dentro da cláusula **UNIQUE**.

- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cpf VARCHAR(11) NULL,
  UNIQUE(matricula, cpf)
);
```

- Podemos definir uma chave única utilizando o comando **ALTER TABLE**.
- **Sintaxe:** `ALTER TABLE tb_nome ADD CONSTRAINT nome_restricao UNIQUE (colunas_sep_vírgula);`

- **Exemplo:**
  - `ALTER TABLE tb_aluno ADD CONSTRAINT uk_aluno UNIQUE(matricula);`
  - `ALTER TABLE tb_aluno ADD CONSTRAINT uk_aluno UNIQUE(matricula, cpf);`
- Podemos remover uma restrição de chave única por meio do comando **ALTER TABLE**.
- **Sintaxe:** `ALTER TABLE tb_nome DROP CONSTRAINT nome_restrição;`
- **Exemplo:** `ALTER TABLE tb_aluno DROP CONSTRAINT uk_aluno;`

Restrições de Integridade

Chave Estrangeira

- Conjunto de colunas de uma tabela cujos valores devem obrigatoriamente corresponder a valores de alguma chave candidata (chave primária ou chave única) de uma outra tabela. Ou seja, faz referência a uma chave candidata de uma outra tabela.
- O banco de dados não pode conter quaisquer valores de chaves estrangeiras não correspondentes. Ou seja, deve evitar que dados inválidos sejam inseridos em uma coluna com esse tipo de restrição.
- O uso de chaves estrangeiras permite mais desempenho, organização e integração do BD, bem como evita a redundância de dados e facilita as consultas.
- Previne contra operações que possam corromper a relação de duas tabelas.

- Utiliza-se a cláusula **FOREIGN KEY** para definir uma chave estrangeira de uma tabela.
- **Exemplo:**

```
CREATE TABLE tb_funcionario (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cod_dep INT,
  CONSTRAINT fk_dep FOREIGN KEY (cod_dep)
     REFERENCES tb_departamento (codigo)
);
```

```sql
CREATE TABLE tb_departamento (
  codigo INT NOT NULL PRIMARY KEY,
  descricao VARCHAR(50) NOT NULL
);

CREATE TABLE tb_funcionario (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cod_dep INT,
  FOREIGN KEY (cod_dep) REFERENCES
     tb_departamento (codigo)
);
```

```
CREATE TABLE tb_funcionario (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cod_dep INT REFERENCES tb_departamento(
     codigo)
);

CREATE TABLE tb_funcionario (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cod_dep INT REFERENCES tb_departamento
);
```

- Podemos definir uma chave estrangeira utilizando o comando **ALTER TABLE**.
- **Sintaxe:** `ALTER TABLE tb_nome ADD CONSTRAINT nome_restrição FOREIGN KEY (colunas_sep_vírgula) REFERENCES tb_nome_referenciada (colunas_sep_vírgula);`
- **Exemplo:** `ALTER TABLE tb_funcionario ADD CONSTRAINT fk_dep FOREIGN KEY (cod_dep) REFERENCES tb_departamento (codigo);`
- Podemos definir algumas ações que o BD tomará para remoção (**ON DELETE**) e atualização (**ON UPDATE**) de registros referenciados por chaves estrangeiras. As principais ações são:
  - **CASCADE:** ao remover ou atualizar um registro com chave referenciada em registros de outras tabelas, a ação será aplicada em todos os registros que contenham essa chave estrangeira;

- **SET NULL:** ao remover ou atualizar um registro com chave referenciada em registros de outras tabelas, todos os valores da chave estrangeira serão definidos como NULL. Todas as colunas de chave estrangeira da tabela de destino (a que referencia, tabela-filha) devem permitir valores nulos para que essa restrição seja executada;
- **RESTRICT:** ao remover ou atualizar um registro com chave referenciada em registros de outras tabelas, a ação será rejeitada para a tabela-pai (a que é referenciada);
- **NO ACTION:** similar ao **RESTRICT**;
- **SET DEFAULT:** ao remover ou atualizar um registro com chave referenciada em registros de outras tabelas, todos os valores da chave estrangeira serão definidos para o valor especificado na cláusula **DEFAULT**, se houver. É reconhecido pelo parser do PostgreSQL, mas é ignorado.

- Quando as cláusulas **ON UPDATE** e **ON DELETE** são omitidas, o tipo de ação default para chaves estrangeiras é sempre **RESTRICT**.
- **Exemplo:**
  - `ALTER TABLE tb_funcionario ADD CONSTRAINT fk_dep FOREIGN KEY (cod_dep) REFERENCES tb_departamento (codigo) ON DELETE CASCADE ON UPDATE NO ACTION;`
  - `ALTER TABLE tb_funcionario ADD CONSTRAINT fk_dep FOREIGN KEY (cod_dep) REFERENCES tb_departamento (codigo) ON DELETE SET NULL;`
- Podemos remover uma restrição de chave estrangeira por meio do comando **ALTER TABLE**.

- Quando as cláusulas **ON UPDATE** e **ON DELETE** são omitidas, o tipo de ação default para chaves estrangeiras é sempre **RESTRICT**.
- **Exemplo:**
  - `ALTER TABLE tb_funcionario ADD CONSTRAINT fk_dep FOREIGN KEY (cod_dep) REFERENCES tb_departamento (codigo) ON DELETE CASCADE ON UPDATE NO ACTION;`
  - `ALTER TABLE tb_funcionario ADD CONSTRAINT fk_dep FOREIGN KEY (cod_dep) REFERENCES tb_departamento (codigo) ON DELETE SET NULL;`

- Podemos remover uma restrição de chave estrangeira por meio do comando **ALTER TABLE**.
- **Sintaxe:** `ALTER TABLE tb_nome DROP FOREIGN KEY nome_restrição;`
- **Exemplo:** `ALTER TABLE tb_funcionario DROP FOREIGN KEY fk_dep;`

- A coluna não permite entrada nula.
- Não pode ser atribuído valor nulo ao atributo.
- Determina a obrigatoriedade de um valor não-nulo a uma coluna.
- Uso da cláusula **NOT NULL**.
- Utilizada nos comando CREATE TABLE e ALTER TABLE.
- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  situacao SMALLINT
);
```

```
CREATE TABLE tb_escola (
  codigo INT NOT NULL PRIMARY KEY,
  razao_social VARCHAR(50) NOT NULL
);
```

- Podemos definir uma restrição de nulidade utilizando o comando ALTER TABLE.
- As principais formas de definir esse tipo de restrição no comando ALTER TABLE são:
  - Adicionar uma nova coluna;
  - Modificar uma coluna existente;
  - Substituir uma coluna existente por uma outra.

- **Exemplo:**
  - `ALTER TABLE tb_aluno ADD COLUMN dt_matricula DATE NOT NULL;`
  - `ALTER TABLE tb_aluno MODIFY COLUMN situacao status_aluno BPCHAR(1) NOT NULL;`
  - `ALTER TABLE tb_aluno MODIFY COLUMN dt_matricula TIMESTAMP NOT NULL;`

- Atribui-se um valor padrão não-nulo para uma coluna quando não existir uma entrada para a mesma.
- Utiliza-se a cláusula DEFAULT para definir esse tipo de restrição em uma tabela.
- Utilizada nos comando CREATE TABLE e ALTER TABLE.
- Aplica-se apenas ao comando INSERT.
- **Exemplo:**

```
CREATE TABLE tb_aluno (
  matricula INT NOT NULL DEFAULT 1,
  nome VARCHAR(50) NOT NULL DEFAULT 'Fulano',
  situacao SMALLINT DEFAULT 1
);
```

```
CREATE TABLE tb_escola (
  codigo INT NOT NULL DEFAULT 1,
  descricao VARCHAR(50) NOT NULL
);
```

- Podemos definir uma restrição **DEFAULT** utilizando o comando **ALTER TABLE**.
- As principais formas de definir esse tipo de restrição no comando **ALTER TABLE** são:
  - Adicionar uma nova coluna;
  - Modificar uma coluna existente;
  - Substituir uma coluna existente por uma outra.

- **Exemplo:**

```
ALTER TABLE tb_aluno ADD COLUMN dt_matricula DATE
    DEFAULT NOW();

ALTER TABLE tb_aluno CHANGE COLUMN situacao
    status_aluno SMALLINT NOT NULL DEFAULT 0;

ALTER TABLE tb_aluno MODIFY COLUMN dt_matricula
    TIMESTAMP NOT NULL DEFAULT NOW();
```

- Seja **RV** uma restrição de verificação para a tabela básica **T**. Então, **T** infringe **RV**, se **T** tem pelo menos uma linha e o valor atual de **T** faz com que a expressão booleana seja **FALSA**.
- Utiliza-se a cláusula CHECK para definir esse tipo de restrição em uma tabela.
- **Sintaxe:** `CHECK <expressão_booleana>`.
- Utilizada nos comando CREATE TABLE e ALTER TABLE.
- Aplica-se apenas aos comandos INSERT e UPDATE.
- Pode ser utilizado em mais de uma coluna.

- **Exemplo:**

```
CREATE TABLE tb_aluno (
   matricula SERIAL NOT NULL PRIMARY KEY,
   nome VARCHAR(50) NOT NULL,
   mensal NUMERIC(10,2) NOT NULL,
   ativo SMALLINT CHECK(ativo IN (0,1)),
   CHECK(mensalidade > 500)
   CONSTRAINT matricula_valida CHECK(matricula > 0)
)
```

- Podemos definir uma restrição CHECK utilizando o comando ALTER TABLE.
- As principais formas de definir esse tipo de restrição no comando ALTER TABLE são:
  - Adicionar uma nova coluna;
  - Modificar uma coluna existente;
  - Substituir uma coluna existente por uma outra.
- **Exemplo:**

```
ALTER TABLE tb_aluno ADD COLUMN dia_pg SMALLINT NOT
    NULL CHECK(dia_pg = 10);
```

```
ALTER TABLE tb_aluno CHANGE COLUMN situacao
    status_aluno SMALLINT NOT NULL CHECK(
    status_aluno IN (0,1));

ALTER TABLE tb_aluno MODIFY COLUMN dt_matricula
    TIMESTAMP NOT NULL CHECK(dt_matricula > DATE(NOW
    ()));
```